

密级：	公开	版本号：	V1.0
使用范围：	客户	文档状态：	发行

HC03 Flutter SDK API 说明

文件编号：	LT-SW-24062402	类别：	RD
拟制：	Chenzq	日期：	2024-6-24
审核：	Yangxd	日期：	2024-6-24
批准：	Tangzp	日期：	2024-6-24



厦门市凌拓通信科技有限公司

■ 文档版本

版本	发布日期	描述
V1.0	2024.06.24	初版

■ 目录

1. 集成说明	4
1.1. 工程开发说明	4
1.2. 集成步骤说明	4
1.2.1. Android 源码集成.....	4
1.2.2. iOS 源码集成.....	4
2. API 说明	5
2.1. 功能说明	5
2.2. 流程图.....	5
2.3. 对外接口详解	5
2.3.1. 接口初始化.....	5
2.3.2. 启动接口.....	6
2.3.3. 停止接口.....	6
2.3.4. 解析设备回调数据接口.....	7
2.3.5. 获取 ECG 数据接口.....	7
2.3.6. 获取血氧数据接口.....	8
2.3.7. 获取血糖数据接口.....	9
2.3.8. 获取血压数据接口.....	10
2.3.9. 获取电池电量数据接口.....	10
2.3.10. 获取体温数据接口	11

1. 集成说明

1.1. 工程开发说明

1. 蓝牙发包和解包以及数据处理封装在lib/src/core的目录下，对外的API在hc03_sdk_impl.dart文件中。
2. 蓝牙库封装在lib/src/bluetooth的目录下，对外的API在bluetooth_manager.dart文件中。
3. 功能的调用，在lib/src/pages_data/health_data.dart文件中。

1.2. 集成步骤说明

1.2.1. Android 源码集成

1. 将android/app/libs/NskAlgoSdk.jar的jar包和android/app/src/main/java/com/ecg文件夹中的java文件和android/app/src/main/jniLibs文件夹中的so库和android/app/src/main/kotlin/com/example/smartring_flutter/EcgPlugin.kt文件移植到你的工程中
2. 在你工程android中的MainActivity的configureFlutterEngine方法中将EcgPlugin初始化，请参考android/app/src/main/kotlin/com/example/smartring_flutter/MainActivity.kt。
AndroidManifest.xml中记得添加权限参考android/app/src/main/AndroidManifest.xml。
android/app/build.gradle中需要添加NskAlgoSdk.jar的依赖以及对jniLibs路径的设置请参考android/app/build.gradle文件。

1.2.2. iOS 源码集成

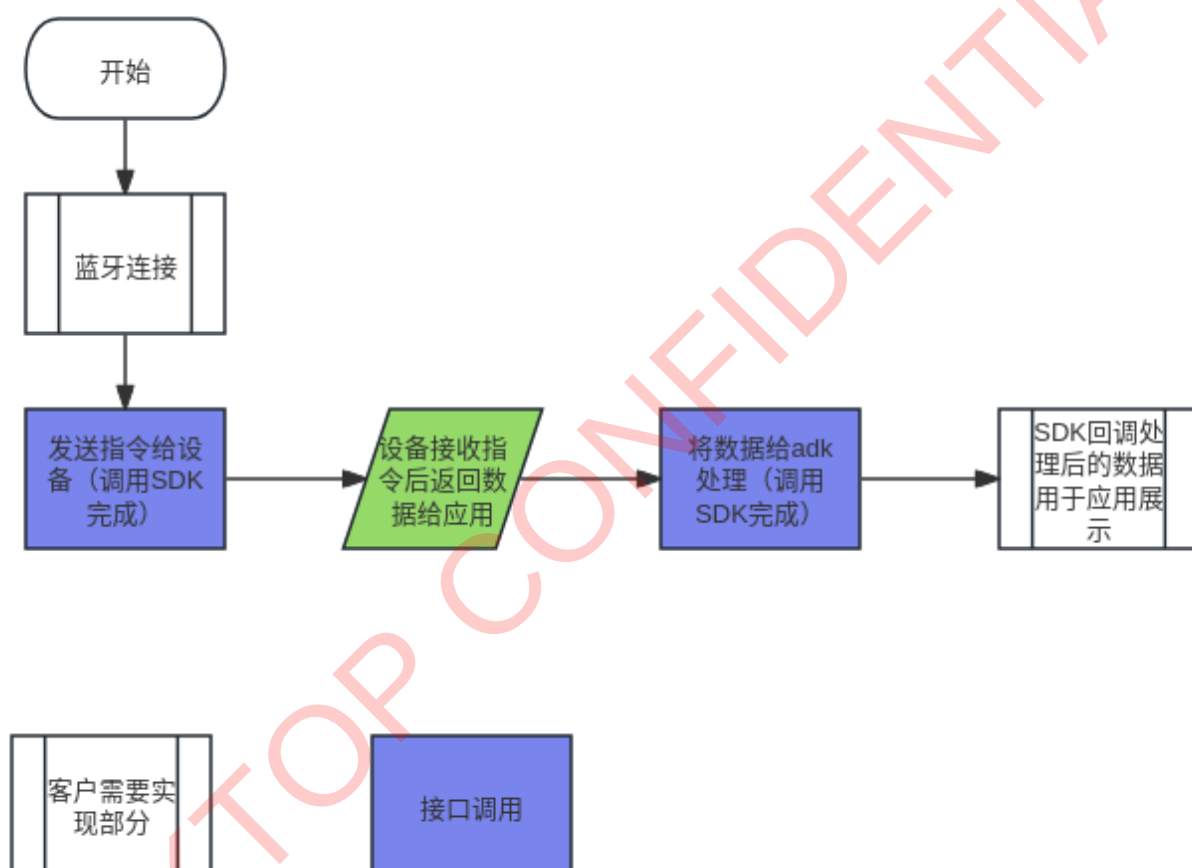
1. 将ios/Runner/libNSKAlgoSDKECG文件夹下的文件和.a库和ios/Runner/SDKHealthMonitor.h, ios/Runner/SDKHealthMonitor.m移植到你的工程中。
注意.a静态库需要在xcode的Target->build Phases->Link Binary With Libraries中添加，然后Target->build Setting->Other Link Flags添加-all_load (.a库在libNSKAlgoSDKECG中)。
2. 在ios/Runner/AppDelegate.h和ios/Runner/AppDelegate.m要对SDKHealthMonitor进行初始化，请参考demo中ios/Runner/AppDelegate.h和ios/Runner/AppDelegate.m。

2. API 说明

2.1. 功能说明

SDK 通过 BLE 与设备建立连接。您可以通过 SDK 轻松与设备进行通信，提取设备提供的心电，血氧，体温数据等服务。

2.2. 流程图



2.3. 对外接口详解

2.3.1. 接口初始化

在你需要使用接口的地方进行初始化

```
Hc03Sdk sdk=Hc03Sdk.getInstance();
```

2.3.2. 启动接口

1) 函数名:

startDetect(Detection detection);

蓝牙连接后调用startDetect启动测试接口

2) 参数:

- Detection.BT: 体温
- Detection.OX: 血氧
- Detection.ECG: 心电
- Detection.BP: 血压
- Detection.BATTERY: 电量
- Detection.BG: 血糖

3) 参考代码:

```
void startDetect(Detection detection) async {  
    var data = sdk.startDetect(detection);  
    await blueToothManager.write(WRITE_UUID, data.toList());  
}
```

2.3.3. 停止接口

1) 函数名:

stopDetect(Detection detection);

调用stopDetect停止测试接口

2) 参数:

- Detection.BT: 体温
- Detection.OX: 血氧
- Detection.ECG: 心电
- Detection.BP: 血压
- Detection.BG: 血糖

3) 参考代码:

```
void stopDetect(Detection detection) async {  
    var data = sdk.stopDetect(detection);  
    await blueToothManager.write(WRITE_UUID, data.toList());  
}
```

2.3.4. 解析设备回调数据接口

- 1) 函数名:

`parseData(data);`

- 2) 参数:

data: 蓝牙回调数据

- 3) 参考代码:

```
blueToothManager.notifyWriteListener(NOTIFY_UUID, (data) async {  
    sdk.parseData(data);  
}, onError: (error) {  
    debugPrint("notifyWriteListener error=$error");    "notify": 0  
});
```

2.3.5. 获取 ECG 数据接口

- 1) 函数名:

`getEcgData;`

- 2) 回调数据类型:

- wave: 用于绘制波形的数据
- HR: 心率数据
- Mood Index: 心情指数
 - 1-20 冷静
 - 21-40 放松
 - 41-60 平衡
 - 61-80 激励
 - 81-100 激动/焦虑/兴奋
- RR: 峰峰值
- HRV: 心率变异性
- RESPIRATORY RATE: 呼吸率
- touch: 手指检测

- 3) 参考代码:

```
final ecgStream = sdk.getEcgData();
ecgStream.listen((data) {
  if (data is EcgData) {
    var message = data.ecg;
    switch (message["type"]) {
      case "wave":
        waveData.add(message["data"] as int);
        if (waveData.length > 2200) {
          waveData.removeAt(0);
        }
        ecgUpdate.value = !ecgUpdate.value;
        break;
      case "HR":
        hr.value = message["value"];
        break;
      case "Mood Index":
        if (hrv.value != 0) {
          stopEcg();
        }
        mood.value = message["value"];
        break;
      case "RR":
        if (message["value"] > maxRR.value) {
          maxRR.value = message["value"];
        } else if (message["value"] < minRR.value) {
          minRR.value = message["value"];
        }
        break;
    }
  }
});
```

2.3.6. 获取血氧数据接口

1) 函数名:

getBloodOxygen;

2) 回调数据类型:

➤ BloodOxygenData:

bloodOxygen: 血氧

heartRate: 心率

➤ FingerDetection: 手指检测

➤ BloodOxygenWaveData: 绘制波形的数据

3) 参考代码:


```
final stream = await sdk.getBloodOxygen();
streamSubscription = stream.data.listen((Hc03BaseMeasurementData data) {
  if (data is BloodOxygenData) {
    bloodOxygen.value = data.bloodOxygen;
    heartRate.value = data.heartRate;
  } else if (data is FingerDetection) {
    debugPrint("fingerDetection=${data.isTouch}");
  } else if (data is BloodOxygenWaveData) {
    hrWaveList.add(data.red.wave.toInt());
    if (hrWaveList.length > 400) {
      hrWaveList.removeAt(0);
    }
    hrWaveUpdate.value = !hrWaveUpdate.value;
  }
}, onError: (Object error, StackTrace stackTrace) {
  if (error is AppException) {
    debugPrint("getTemperature exception=${error.message}");
  }
});
stream.stop.listen((Hc03BaseMeasurementData data) {
  if (data is StopMeasure) {
    stopBloodOxygen();
  }
});
```

2.3.7. 获取血糖数据接口

1) 函数名:

getBloodGlucoseData;

2) 回调数据类型:

- BloodGlucoseSendData: 需要发送给设备的数据
- BloodGlucosePaperState: 测量过程中血糖试纸状态信息
- BloodGlucosePaperData: 血糖数据

3) 参考代码:

```
sdk.getBloodGlucoseData().listen((data) {
  if (data is BloodGlucoseSendData) {
    sendCmd(data.sendList);
  } else if (data is BloodGlucosePaperState) {
    bloodGlucose.value = data.message;
  } else if (data is BloodGlucosePaperData) {
    bloodGlucose.value = "blood glucose: ${data.data} ";
  }
});
```

2.3.8. 获取血压数据接口

1) 函数名:

getBloodPressureData;

2) 回调数据类型:

- BloodPressureSendData: 需要发送给设备的数据
- BloodPressureProcess: 显示血压测量的流程
- BloodPressureResult: 血压数据

ps: 收缩压

pd: 舒张压

hr: 心率

3) 参考代码:

```
sdk.getBloodPressureData().listen((data) {  
  if (data is BloodPressureSendData) {  
    sendCmd(data.sendList);  
  } else if (data is BloodPressureProcess) {  
    debugPrint("BloodPressureProcess=${data.process}");  
  } else if (data is BloodPressureResult) {  
    bloodPressure.value =  
      "SystolicBloodPressure:${data.ps}mmHg DiastolicBloodPressure:${data.pd}mmHg hr:${data.hr}times/minute";  
    debugPrint(  
      "BloodPressureResult ps=${data.ps} pd=${data.pd} hr=${data.hr}");  
  }  
}, onError: (Object error, StackTrace stackTrace) {  
  if (error is AppException) {  
    debugPrint("getBloodPressureData exception=${error.message}");  
    bloodPressure.value = error.message;  
  }  
});
```

2.3.9. 获取电池电量数据接口

1) 函数名:

getBattery;

2) 回调数据类型:

- BatteryLevelData: 电量百分比数据
- BatteryChargingStatus: 电池充电状态

3) 参考代码:

```
sdk.getBattery().then((data) {  
  if (data is BatteryLevelData) {  
    battery.value = "battery: ${data.batteryLevel}%";  
  } else if (data is BatteryChargingStatus) {  
    battery.value = data.batteryCharging ? "charging" : "unCharging";  
  }  
});
```

2.3.10. 获取体温数据接口

1) 函数名:

getTemperature;

2) 回调数据类型:

TemperatureData: 体温数据

3) 参考代码:

```
void startTemperature() {  
  startDetect(Detection.BT);  
  sdk.getTemperature().then((Hc03BaseMeasurementData data) {  
    if (data is TemperatureData) {  
      temperature.value = data.temperature;  
    }  
  }).catchError((error) {  
    if (error is AppException) {  
      debugPrint("getTemperature exception=${error.message}");  
    }  
  });  
}
```